

FSC LEGAL OS

Documento de Onboarding & Checklist da Equipe — Fábio Cunha Advocacia (OAB/RO 10.849)

Bem-vindo(a) ao time! Este documento é o seu mapa do projeto: apresenta a plataforma, o que já está no ar, o que está em ajuste, o que está em implementação e o que vem pela frente. Leia do início ao fim antes de tocar em qualquer código. Estamos construindo algo grande — e com responsabilidade (ética da OAB sempre).

Documento de referência — nada aqui foi executado automaticamente; é o nosso alinhamento de arquitetura e roadmap.

1. O que é o FSC Legal OS

Plataforma de **advocacia digital** do escritório Fábio Cunha Advocacia (Dr. Fábio Silva Cunha, OAB/RO 10.849). Automatiza a jornada do cliente ponta a ponta:

```
Lead (site / WhatsApp)
-> Triagem identifica a AREA -> Agente Especialista assume
-> Acolhe + valida a tese citando o entendimento dos tribunais
-> Fecha o contrato (ZapSign) -> Cobrança (Asaas)
-> Coleta documentos e provas -> Analise -> Peticao
-> Advogado revisa no CRM -> "Aprovar e Protocolar" -> RPA protocola
-> Cliente acompanha as fases pela plataforma (estilo "rastreamento de entrega")
```

Diferencial: um Agente de IA humanizado (espelha a personalidade do Dr. Fábio: educado, atencioso, cristão, foco na dor do cliente) que conduz da triagem ao fechamento, e um Robô de Jurisprudência que se atualiza sozinho.

2. Arquitetura técnica (stack e endereços)

Camada	Tecnologia	Onde
Frontend	Next.js 14 + Tailwind	app.fscadvocaciadigital.com.br
Backend/API	FastAPI (Python)	api.fscadvocaciadigital.com.br
Banco/Auth/Storage	Supabase	projeto midywtbybplbdqmkxfoj
Servidor	Hetzner VPS CPX22 (Ubuntu 24.04)	IP 159.69.124.171 (Falkenstein)
Proxy/SSL	Caddy (Let's Encrypt)	no servidor
Containers	Docker: api, worker, redis, frontend	/opt/fsc-legal-os/backend
Código	GitHub advfabioscunha-design/fsc-legal-os	branch main (privado)
DNS	HostGator (A: api/app -> IP)	—

Estrutura do repositório (resumo)

```
backend/  app/ (main.py, agentes/, integracoes/, core/), workers/, jobs/, docker-compose.yml
frontend/ app/ (page, entrar, cliente, crm, equipe, portal, components), lib/, Dockerfile
supabase/ migrations/ (0001..0004), seed_teses.py
infra/    caddy/Caddyfile, deploy.sh
```

Fluxo de deploy atual

- Editar o código → SUBIR_GITHUB.bat (push para o GitHub).
- No servidor (SSH root@159.69.124.171): cd /opt/fsc-legal-os && git pull.
- cd backend && docker compose up -d --build api frontend.

3. Status geral — Roadmap

FEITO E VALIDADO

- Infraestrutura Cloud (VPS Hetzner) + Docker + Caddy/SSL.
- Banco, Auth e Storage (Supabase) com migrations 0001–0004.
- Frontend (Next.js) no ar com HTTPS.
- Motor de pesquisa e ingestão de Jurisprudência (DataJud/CNJ) — radar semanal automático.
- Autenticação por perfis: Cliente e Operador (RLS, código de acesso da equipe).
- Área do Cliente: triagem do caso + acompanhamento das fases (estilo rastreio).
- CRM do Operador (esteira/kanban) — validado com casos reais de teste.
- Cadastro de operadores (/equipe) com código de acesso (multiusuário).
- Validação de CPF (dígitos verificadores) no cadastro/triagem.
- Agente Web com a personalidade do Dr. Fábio + funil de fechamento de 4 passos.
- Botão flutuante de WhatsApp no site (número 48 98835-7992).

FEITO, MAS EXIGE TESTES/AJUSTES

- Responsividade do Frontend (Tailwind sm/md/lg) — ver item 4.1.
- Agressividade do Agente Web para fechamento — ver item 4.2 (prompt no Anexo A).
- Integração Asaas (cobrança) e ZapSign (assinatura): código pronto, faltam as chaves de API.

EM IMPLEMENTAÇÃO (FOCO ATUAL)

- WhatsApp Cloud API (Meta) — app/verificação em andamento (parou na verificação da conta de desenvolvedor pela Central de Contas).
- Handoff Web → WhatsApp com contexto compartilhado (Session ID) — ver Anexo B.
- Whisper (transcrição de áudio) + ElevenLabs (voz clonada do Dr. Fábio).

PENDENTE (BACKLOG — PRÓXIMAS SPRINTS)

- Gatilhos de vídeos de onboarding (Boas-vindas, Recurso, Execução).
- Módulo E-commerce de Contratos (self-service) — ver item 4.4.
- Leitura multimodal (Vision para PDF/imagem/vídeo) no WhatsApp.
- Certificado digital A3 em nuvem para o RPA de protocolo.

4. Instruções e especificações (na ordem de execução)

4.1 Correção de rota — Responsividade e UI/UX

- O site (Next.js) deve ser 100% responsivo com Tailwind (sm: md: lg:).
- Revisar todas as páginas (home, entrar, cliente, crm, equipe, portal): grids, fontes, paddings.
- O kanban do CRM (hoje largo) deve se adaptar a mobile (rolagem/coluna única) e desktop sem quebrar.
- Meta: legibilidade perfeita do celular ao monitor.

4.2 Correção do Agente Web — Gatilho de Fechamento

- O bot está 'enrolando'. Aplicar a regra restrita de fechamento (texto completo no Anexo A).
- Essência: ao identificar a tese, OBRIGATORIAMENTE parar de perguntar, nomear a ação, coletar só Nome e CPF e encaminhar o link. Não estender a conversa.

4.3 Arquitetura do Agente WhatsApp (State Machine)

- Contexto compartilhado: ler o banco e continuar de onde o Agente Web parou; nunca repetir perguntas (Anexo B).
- SLA de resposta: timeout máximo de 20 segundos para a IA responder.
- Espelhamento de mídia: texto → texto; áudio → resposta em áudio com a voz clonada (ElevenLabs).
- Leitura multimodal: Whisper transcreve áudios longos; Vision lê PDF/imagem e interpreta vídeo, devolvendo resumo ('Li o documento / assisti ao vídeo. Entendi que a situação é X. Confirma?').
- Fluxo pós-contrato: após enviar o ZapSign, perguntar 'Conseguiu acessar e assinar o contrato?'. Quando o webhook do ZapSign confirmar → dispara o Vídeo 1 de Boas-vindas e só então inicia a coleta de documentos (RG, comprovantes...).

4.4 Novo módulo — E-commerce de Contratos (Self-Service) [BACKLOG]

- Venda de contratos avulsos na plataforma.
- Preços base: R\$ 89,90 (Baixa) · R\$ 129,90 (Média) · R\$ 297,00 (Alta Complexidade).
- Regra de redação por IA: o Agente Redator deve alertar sobre cláusulas abusivas. Se o cliente insistir, incluir cláusula de isenção de responsabilidade técnica do escritório.
- O PDF só é liberado para aprovação/assinatura após o pagamento na plataforma.

5. Acessos (sem expor segredos)

- Servidor: SSH root@159.69.124.171 (senha definida pelo Dr. Fábio).
- Segredos ficam em /opt/fsc-legal-os/backend/.env no servidor (NUNCA no GitHub): Supabase Service Key, Claude API Key, EQUIPE_CODIGO, WhatsApp (a configurar), Asaas/ZapSign (a configurar).
- Modelo de IA atual: claude-sonnet-4-6 (atendimento) e claude-haiku-4-5 (tarefas rápidas).
- Código da equipe (operador): definido em EQUIPE_CODIGO no .env.

Anexo A — System Prompt ajustado do Agente Web (gatilho de fechamento)

Regra a ser somada/ajustada no gerar_system_prompt (backend/app/agentes/especialista.py), reforçando o funil já existente:

GATILHO DE FECHAMENTO (OBRIGATORIO): assim que o cliente relatar o problema e voce identificar a tese aplicavel, e PROIBIDO continuar perguntando. Faca exatamente:

- 1) "Entendi perfeitamente seu caso. A solucao ideal e a [Nome da Acao]. Para nao perdermos tempo, vou preparar o seu contrato."
- 2) Colete SOMENTE Nome e CPF (nada alem disso agora).
- 3) Encaminhe o link do contrato e finalize com a mensagem de transicao para o WhatsApp.

NAO estenda a conversa, nao peca documentos, nao faca perguntas extras nesta fase.

Limites eticos (OAB): nunca minta, nunca garanta resultado, respeite a decisao do cliente.

Anexo B — Lógica FastAPI: contexto compartilhado Web → WhatsApp (Session ID)

Princípio: a 'sessão' é o próprio CASO no banco (casos), vinculado ao cliente. O Agente Web e o Agente WhatsApp usam o MESMO histórico (mensagens) e o MESMO agente especialista — a conversa continua sem repetir perguntas.

```
# backend/app/integracoes/whatsapp.py (referencia)
def processar_webhook(payload: dict) -> dict:
    msg = payload["entry"][0]["changes"][0]["value"]["messages"][0]
    numero = msg["from"] # telefone = chave da sessao
    texto = extrair_texto_ou_transcrever(msg) # whisper se for audio

    db = get_db()
    # 1) acha o cliente pelo telefone (mesmo cadastro do site)
    cli = db.table("clientes").select("id") \
        .or_(f"whatsapp.eq.{numero},telefone.eq.{numero}") \
        .maybe_single().execute().data
    if cli:
        # 2) recupera o CASO ativo (a "sessao" iniciada no site)
        caso = db.table("casos").select("id") \
            .eq("cliente_id", cli["id"]) \
            .not_.in_("estado", ["CONCLUIDO", "CANCELADO", "INVIAVEL"]) \
            .order("criado_em", desc=True).limit(1).execute().data
        if caso:
            # 3) MESMO agente, MESMO historico -> continua de onde parou
            return especialista.atender(caso[0]["id"], texto, canal="WHATSAPP")
    # 4) numero novo -> triagem cria a sessao e o especialista responde
    return triagem.criar_caso(nome=f"Lead {numero[-4:]}", contato=numero,
                             relato=texto, canal="WHATSAPP")
```

Pontos-chave:

- O histórico vem de `_historico(caso_id)` (tabela mensagens) → o agente já 'lembra' tudo.
- O system prompt injeta os dados já conhecidos do cliente (nome, CPF) → não repete perguntas.
- SLA 20s: envolver a chamada do Claude em `asyncio.wait_for(..., timeout=20)`; no timeout, mandar mensagem-ponte e processar em background.
- Espelhamento: `responder_espelhando(numero, texto, cliente_mandou_audio)` → texto ou áudio (ElevenLabs).
- Pós-contrato: o webhook do ZapSign muda o estado → dispara Vídeo 1 e abre a coleta de docs.

Entendimento da arquitetura (confirmação)

A plataforma é um cérebro único (banco + agente especialista) acessado por dois canais (Web e WhatsApp). A 'sessão' é o caso do cliente; ambos os canais leem e escrevem no mesmo histórico, garantindo continuidade sem repetição. As próximas frentes são: responsividade, gatilho de fechamento mais firme, WhatsApp Cloud API + handoff com contexto, mídia (Whisper/ElevenLabs/Vision) e o módulo

de contratos self-service.

Próximo passo imediato: concluir a verificação da conta de desenvolvedor na Meta para obter WHATSAPP_TOKEN e Phone Number ID e ligar o webhook <https://api.fscadvocadiadigital.com.br/webhooks/whatsapp> (verify token: fsc-legal-os).